

Phase 3 — Exercices pour aller plus loin

Cinq exercices avancés sur les agents : tools dynamiques, LangSmith, sous-LLM, streaming

Cours PyTutor2

Résumé

Ce document complète `phase3.tex` avec cinq exercices avancés sur les agents : ajout d'un tool à la volée, observabilité dans LangSmith, paramètres optionnels exposés au LLM, pattern *tool qui appelle un sous-LLM*, et streaming en deux modes (`updates` et `messages`). Chaque exercice suit la même structure que les principaux.

Table des matières

1	Ajouter un cinquième tool à l'agent	2
1.1	Solution	2
1.1.1	Architecture	2
1.1.2	Le code complet	2
1.1.3	Explications pas à pas	3
1.2	Vérification dans LangSmith	5
2	LangSmith pour observer un agent	5
2.1	Solution	6
2.1.1	Architecture	6
2.1.2	Le code complet	6
2.1.3	Explications pas à pas	7
2.2	Vérification dans LangSmith	8
3	Tool avec paramètre <code>timeout</code> configurable	8
3.1	Solution	9
3.1.1	Architecture	9
3.1.2	Le code complet	9
3.1.3	Explications pas à pas	10
3.2	Vérification dans LangSmith	12
4	Tool qui appelle un sous-LLM (<code>evaluate_student_answer</code>)	13
4.1	Solution	14
4.1.1	Architecture	14
4.1.2	Le code complet	14
4.1.3	Explications pas à pas	15
4.2	Vérification dans LangSmith	18
5	Streaming avec <code>agent.stream()</code>	19
5.1	Solution	19
5.1.1	Architecture	19
5.1.2	Le code complet	19
5.1.3	Explications pas à pas	20
5.2	Vérification dans LangSmith	22

1 Ajouter un cinquième tool à l'agent

Exercice bonus 1. Étendre l'inventaire de tools en une fonction

Écris un tool `current_python_version()` qui renvoie la version de Python utilisée par le processus, et ajoute-le à l'agent PyTutor. Vérifie ensuite avec une question genre « *Quelle version utilises-tu ? Est-ce que je peux utiliser `match/case` ?* ».

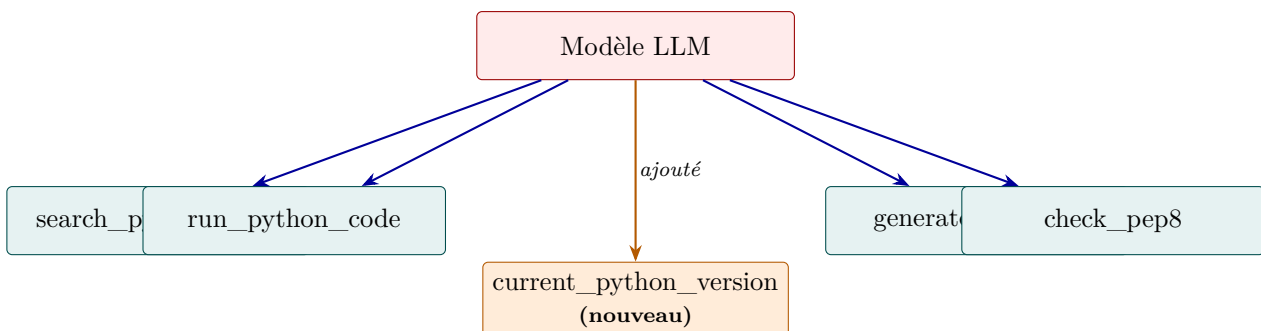
Contrainte : la docstring du tool doit mentionner les cas d'usage typiques (compatibilité, features par version : `match` en 3.10, `ExceptionGroup` en 3.11, etc.) pour que le LLM sache *quand* l'appeler.

Objectifs pédagogiques

- Mesurer la **trivialité** de l'ajout d'un tool une fois que l'agent est construit : une fonction `@tool`, et on l'ajoute à la liste.
- Comprendre que le modèle découvre le nouveau tool **automatiquement** via son nom et sa docstring — pas de configuration explicite.
- Voir le rôle des cas d'usage dans la docstring : c'est ce qui déclenche (ou pas) l'appel du tool selon la question.

1.1 Solution

1.1.1 Architecture



Un cinquième tool s'ajoute sans modification du reste : même `system_prompt`, même `model`, même architecture LangGraph. Le modèle, à l'invocation, voit 5 tools dans son contrat au lieu de 4.

1.1.2 Le code complet

```
1 import sys
2 from langchain.agents import create_agent
3 from langchain_core.tools import tool
4 from langchain_core.messages import HumanMessage
5 from config import get_model
6 from phase3_agent import TOOLS, SYSTEM_PROMPT, print_trace
7
8
9 @tool
10 def current_python_version() -> str:
11     """Renvoie la version Python du processus qui exécute PyTutor.
12
13     À utiliser quand l'élève pose des questions sur la compatibilité, les
14     features récentes (match/case en 3.10, ExceptionGroup en 3.11, etc.),
15     ou simplement quand il demande quelle version est utilisée.
```

```

16     """
17     return f"Python {sys.version.split()[0]} ({sys.platform})"
18
19
20 def exercise_1() -> None:
21     # On rajoute simplement le nouveau tool à la liste existante.
22     agent = create_agent(
23         model=get_model(),
24         tools=TOOLS + [current_python_version],
25         system_prompt=SYSTEM_PROMPT,
26     )
27
28     result = agent.invoke({
29         "messages": [HumanMessage(content=(
30             "Quelle version de Python utilises-tu pour exécuter mon code ? "
31             "Est-ce que je peux utiliser le pattern matching avec `match` / `case` ?"
32         ))]
33     })
34     print_trace(result["messages"])

```

1.1.3 Explications pas à pas

Étape 1 — La fonction Python décorée

Un tool n'a besoin que de trois choses : un décorateur `@tool`, une signature typée, une docstring :

```

1 @tool
2 def current_python_version() -> str:
3     """Renvoie la version Python..."""
4     return f"Python {sys.version.split()[0]} ({sys.platform})"

```

Sans argument : la signature `() -> str` est minimale. Le LLM n'a rien à fournir en entrée, juste à décider d'appeler le tool.

Étape 2 — L'ajout à la liste tools

```

1 tools=TOOLS + [current_python_version]

```

C'est tout. Aucun re-enregistrement, aucune configuration. Le nouveau tool est passé à `create_agent` qui l'injecte dans le contrat du LLM.

***Pattern de composition** : importer la liste `TOOLS` de `phase3_agent.py` et la combiner avec de nouveaux tools évite de dupliquer la définition de l'agent. C'est l'équivalent d'une extension en orienté-objet, mais en plus simple : juste de la concaténation de listes.*

Étape 3 — La docstring déclenche l'usage

La docstring liste des cas d'usage spécifiques :

- « questions sur la compatibilité » — déclenche l'appel quand l'utilisateur demande s'il peut utiliser `walrus` ou autre feature récente.
- « features récentes (match/case en 3.10...) » — donne au LLM des *exemples concrets* de questions qui doivent y mener.
- « ou simplement quand il demande quelle version » — couvre le cas direct.

Étape 4 — Le comportement attendu de l'agent

Sur la question test : « *Quelle version de Python utilises-tu ? Est-ce que je peux utiliser `match` / `case` ?* »

L'agent devrait :

1. Appeler `current_python_version()`.
2. Observer la sortie (par exemple "Python 3.11.4 (darwin)").
3. Répondre en mentionnant que `match/case` (PEP 634) est disponible depuis Python 3.10, et donc utilisable.

Si l'agent n'appelle PAS le tool, c'est que la docstring n'est pas assez explicite. Reformule-la pour ajouter le mot-clé qui correspond à la question (par exemple, ajouter « `pattern matching`, `match`, `case` » parmi les cas d'usage).

Sortie de l'exécution

```
=====
EXERCICE 1 -- Ajout d'un tool current_python_version()
=====

[USER]:
  Quelle version de Python utilises-tu pour exécuter mon code ?
  Est-ce que je peux utiliser le pattern matching avec
  `match` / `case` ?

[AI]:
  Je vais vérifier la version de Python utilisée.
  -> tool_use: current_python_version({})

[TOOL current_python_version]:
  Python 3.12.7 (darwin)

[AI]:
  Excellente nouvelle !

  Je utilise **Python 3.12.7**, ce qui signifie que **oui, tu peux
  tout à fait utiliser le pattern matching avec `match` / `case`** !

  Le pattern matching a été introduit en **Python 3.10**, donc tu as
  accès à cette fonctionnalité. Voici un petit exemple pour te
  montrer que ça fonctionne :

  -> tool_use: run_python_code({'code':
    'def classifieur_nombre(n):\n'
    '    match n:\n'
    '        case 0: return "zéro"\n'
    '        case 1: return "un"\n'
    '        case 2: return "deux"\n'
    '        case _: return "autre nombre"\n'
    'print(classifieur_nombre(0))\n'
    'print(classifieur_nombre(1))\n'
    'print(classifieur_nombre(5))'})

[TOOL run_python_code]:
  STDOUT:
  zéro
  un
  autre nombre
  Exit code: 0

[AI]:
  Parfait ! Comme tu vois, le pattern matching fonctionne sans
```

```
problème. Tu peux utiliser :
- Les patterns simples (valeurs littérales)
- Les patterns structurels (tuples, listes, dictionnaires)
- Les patterns avec gardes (conditions supplémentaires
  avec `if`)
- Le wildcard `_` pour capturer n'importe quoi
```

1.2 Vérification dans LangSmith

L'arbre d'exécution doit montrer :

```
agent (LLM, tour 1)
  Output : tool_calls = [{current_python_version}]
    tools (ToolNode)
      current_python_version() → "Python 3.11.4..."
agent (LLM, tour 2)
  Output : "J'utilise Python 3.11. match/case est..."
```

Vérifie aussi l'*Input* du premier appel LLM : les **5 tools** doivent être listés dans le schéma envoyé, dont `current_python_version` avec sa docstring.

Récapitulatif

Ce qu'il faut retenir :

1. Ajouter un tool est **trivial** : une fonction `@tool`, et on l'append à la liste passée à `create_agent`.
2. Pattern de composition : `tools=TOOLS + [new_tool]` évite la duplication.
3. La docstring est le **seul** levier pour orienter le modèle sur le quand-utiliser. Soigner les cas d'usage explicites.
4. Si le modèle n'appelle pas le tool attendu : c'est presque toujours un problème de docstring, pas de logique.

2 LangSmith pour observer un agent

Exercice bonus 2. Tracer la boucle ReAct dans l'UI LangSmith

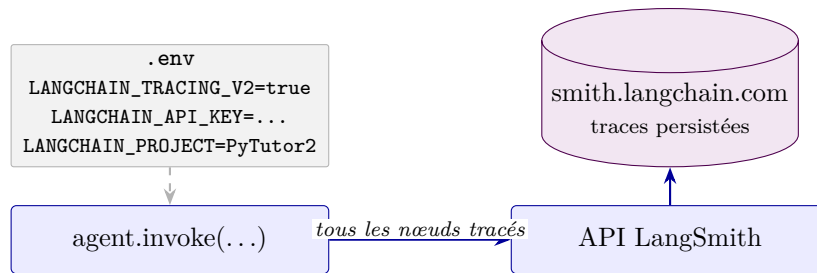
Configure LangSmith (variables d'environnement `LANGCHAIN_TRACING_V2`, `LANGCHAIN_API_KEY`, `LANGCHAIN_PROJECT`) et exécute l'agent sur une question **multi-tool** pour obtenir une trace riche. Ouvre la trace dans l'UI et explore l'arbre d'exécution.
Contrainte : l'exercice doit vérifier programmatiquement la présence des variables d'env et afficher un mode d'emploi clair si la config manque.

Objectifs pédagogiques

- Configurer LangSmith pour le suivi d'un agent (variables spécifiques `LANGCHAIN_*`).
- Lire l'arbre d'exécution d'un agent dans l'UI : nœuds `agent` / `tools`, prompts exacts, tokens consommés, latence.
- Comprendre que *l'agent n'a pas de mémoire magique* : c'est l'historique qui s'accumule et est repassé au modèle à chaque tour. LangSmith le rend visible.

2.1 Solution

2.1.1 Architecture



Quand `LANGCHAIN_TRACING_V2=true` et qu'une clé API valide est présente, *chaque* Runnable et *chaque* appel LLM sont automatiquement envoyés à LangSmith. Aucun changement de code nécessaire.

2.1.2 Le code complet

```
1 import os
2 from langchain_core.messages import HumanMessage
3 from phase3_agent import build_agent
4
5
6 def exercise_2_langsmith() -> None:
7     # --- Vérification de la configuration ---
8     tracing = os.getenv("LANGCHAIN_TRACING_V2", "").lower() in {"true", "1"}
9     api_key = os.getenv("LANGCHAIN_API_KEY")
10    project = os.getenv("LANGCHAIN_PROJECT", "default")
11
12    print(f"LANGCHAIN_TRACING_V2 : {'V' if tracing else 'X'}")
13    print(f"LANGCHAIN_API_KEY      : {'V' if api_key else 'X'}")
14    print(f"LANGCHAIN_PROJECT       : {project}")
15
16    if not (tracing and api_key):
17        print("-> Configure ces variables dans ton .env puis relance.")
18        return
19
20    # --- Exécution d'un agent multi-tool pour une trace riche ---
21    agent = build_agent()
22    agent.invoke({
23        "messages": [HumanMessage(content=(
24            "Cherche dans la doc ce qu'est un dictionnaire en Python, "
25            "puis génère un exercice débutant sur ce sujet, "
26            "et vérifie que le code de référence est PEP 8 compliant."
27        ))]
28    })
29    print(f"-> Trace envoyée. Ouvre https://smith.langchain.com (projet '{project}').")
```

2.1.3 Explications pas à pas

Étape 1 — Les variables d'environnement LangSmith

Variable	Rôle
LANGCHAIN_TRACING_V2	"true" pour activer l'envoi automatique des traces. Sans ça, les chaînes/agents s'exécutent normalement, rien n'est envoyé.
LANGCHAIN_API_KEY	Clé personnelle créée sur <code>smith.langchain.com</code> (commence par <code>lsv2_pt_...</code>).
LANGCHAIN_PROJECT	Nom du projet où grouper les traces. Ex. : "PyTutor2".
LANGCHAIN_ENDPOINT	Optionnel. <code>https://eu.api.smith.langchain.com</code> pour un compte EU.

***Note de nommage** : les variables historiques sont préfixées `LANGCHAIN_` ; la nouvelle version unifiée est préfixée `LANGSMITH_`. Les deux sont reconnues. Cohérence avec le reste du projet recommandée.*

Étape 2 — Une question multi-tool pour une trace riche

Pour avoir une trace pédagogiquement intéressante, choisis une question qui force l'agent à enchaîner plusieurs tools :

« Cherche dans la doc ce qu'est un dictionnaire en Python, puis génère un exercice débutant sur ce sujet, et vérifie que le code de référence est PEP 8 compliant. »

Cette question demande explicitement :

1. `search_python_docs` (« cherche dans la doc »).
2. `generate_exercise` (« génère un exercice »).
3. `check_pep8` (« vérifie PEP 8 »).

L'agent va donc faire **3 itérations** de la boucle ReAct, ce qui donne une trace très lisible.

Étape 3 — Ce qu'on voit dans l'UI LangSmith

Dans `smith.langchain.com`, dans ton projet :

1. Une trace nommée **LangGraph** apparaît : l'agent *est* un graphe LangGraph.
2. L'**arbre d'exécution** montre l'alternance **agent** (LLM) / **tools** (ToolNode) sur 3 cycles.
3. Chaque **agent** (model call) expose :
 - *Input* : système + historique + schémas des tools.
 - *Output* : la décision (`tool_calls`) ou la réponse finale.
4. Chaque **tools** expose les noms des tools appelés, leurs arguments, et leurs sorties intégrales.
5. Tokens et latence *par appel* sont visibles, agrégés sur la trace racine.

Étape 4 — L'insight clé : l'historique grossit

*À chaque tour du modèle, regarde l'onglet Input dans la trace. Tu vois **tout l'historique précédent** qui est repassé au LLM. L'agent n'a pas de mémoire magique : c'est juste l'historique qui grossit, et que LangChain repasse à chaque appel. C'est aussi pourquoi le coût en tokens d'entrée augmente à chaque tour.*

2.2 Vérification dans LangSmith

(Cet exercice EST la vérification.)

Quatre choses à pointer dans la trace pour bien comprendre :

- L'**alternance** `agent` $\rightarrow$ `tools` $\rightarrow$ `agent` $\rightarrow$...
- Le *même* `system_prompt` en tête de chaque appel `agent` (constant).
- L'*historique qui grandit* dans l'input de chaque appel `agent`.
- La **dernière** étape : un `agent` qui produit un Output **sans** `tool_calls` — c'est la réponse finale.

Récapitulatif

Ce qu'il faut retenir :

1. 3 variables d'env : `LANGCHAIN_TRACING_V2=true`, `LANGCHAIN_API_KEY=...`, `LANGCHAIN_PROJECT=...`
2. Tracing automatique : aucune modification de code une fois activé.
3. Une trace d'agent montre l'**alternance** `agent/tools/agent...` jusqu'à la décision finale.
4. L'historique grossit visible dans l'Input de chaque appel LLM successif. C'est l'observation pédagogique fondamentale.
5. LangSmith expose `tokens` et `latence` par appel, agrégés sur la trace racine : indispensable pour optimiser un agent.

3 Tool avec paramètre `timeout` configurable

Exercice bonus 3. Exposer un paramètre par défaut au modèle

Modifie `run_python_code` pour accepter un paramètre `timeout_seconds: int = 5`. La docstring doit guider le modèle sur *quand* surcharger la valeur par défaut (calcul long manifeste) et *quand pas* (la majorité des cas). Teste avec deux scénarios : un code rapide et un code volontairement long.

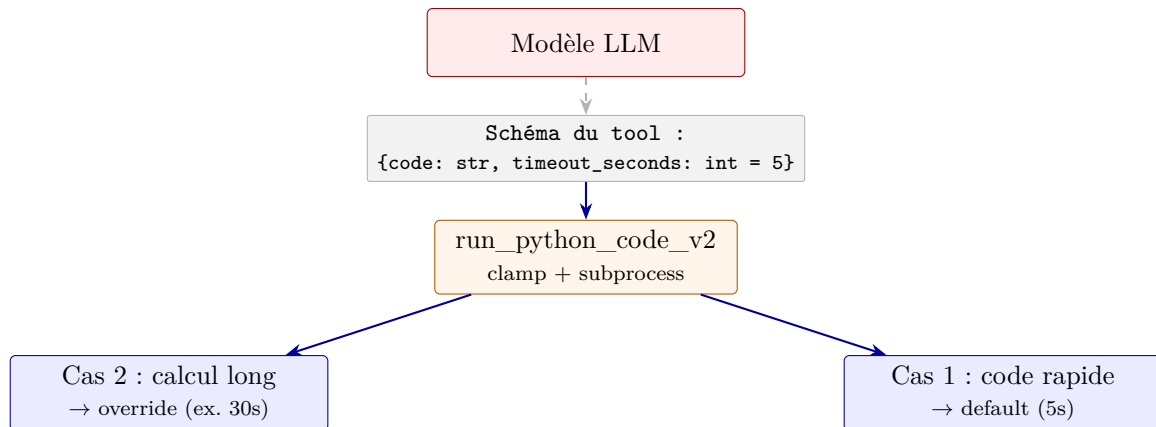
Contrainte : le timeout doit être *clampé* côté serveur ($1 \leq t \leq 60$ secondes) pour empêcher des valeurs absurdes.

Objectifs pédagogiques

- Comprendre que les paramètres avec valeur par défaut sont **exposés au modèle** dans le schéma JSON du tool.
- Voir comment la docstring conditionne la décision du modèle d'override ou non.
- Mettre en place une **défense en profondeur** : une consigne dans la docstring *plus* un clamping côté code.

3.1 Solution

3.1.1 Architecture



Le modèle voit la signature complète, lit la docstring, et décide à chaque appel s'il garde la valeur par défaut ou la surcharge.

3.1.2 Le code complet

```
1 import subprocess
2 import sys
3 import tempfile
4 from pathlib import Path
5
6 from langchain_core.tools import tool
7
8
9 @tool
10 def run_python_code_v2(code: str, timeout_seconds: int = 5) -> str:
11     """Exécute du code Python et renvoie stdout + stderr.
12
13     Args:
14         code: Le code Python à exécuter, complet et autonome.
15         timeout_seconds: Délai max avant interruption. Défaut : 5 secondes.
16             Augmente cette valeur SEULEMENT si le code requiert manifestement
17             un calcul long (entraînement, traitement de gros datasets, etc.).
18             Maximum raisonnable : 60 secondes.
19     """
20     timeout_seconds = max(1, min(timeout_seconds, 60)) # clamp défensif
21
22     with tempfile.NamedTemporaryFile(mode="w", suffix=".py", delete=False) as f:
23         f.write(code)
24         path = f.name
25     try:
26         result = subprocess.run(
27             [sys.executable, path],
28             capture_output=True, text=True, timeout=timeout_seconds,
29         )
30         parts = []
31         if result.stdout:
32             parts.append(f"STDOUT:\n{result.stdout}")
33         if result.stderr:
34             parts.append(f"STDERR:\n{result.stderr}")
35         parts.append(f"Exit code: {result.returncode} (timeout utilisé:
36             ↪ {timeout_seconds}s)")
```

```

36     return "\n\n".join(parts)
37 except subprocess.TimeoutExpired:
38     return (
39         f"ERREUR: timeout après {timeout_seconds} secondes. "
40         "Si tu pensais que le code devait être long, augmente timeout_seconds. "
41         "Sinon, il y a sans doute une boucle infinie ou une récursion non terminale."
42     )
43 finally:
44     Path(path).unlink(missing_ok=True)

```

3.1.3 Explications pas à pas

Étape 1 — Le paramètre avec valeur par défaut

```

1 def run_python_code_v2(code: str, timeout_seconds: int = 5) -> str:

```

Le décorateur `@tool` extrait *toute* la signature et la traduit en JSON Schema :

```

1 {
2     "type": "object",
3     "properties": {
4         "code": {"type": "string"},
5         "timeout_seconds": {"type": "integer", "default": 5},
6     },
7     "required": ["code"],      # timeout_seconds N'EST PAS requis
8 }

```

Ce JSON Schema est envoyé au LLM. Le modèle peut donc **choisir** de remplir `timeout_seconds`, ou de laisser le tool utiliser le défaut.

Étape 2 — La docstring guide la décision

C'est la docstring du paramètre qui pilote le comportement :

*« Augmente cette valeur **SEULEMENT** si le code requiert manifestement un calcul long (entraînement, traitement de gros datasets, etc.). Maximum raisonnable : 60 secondes. »*

Trois leviers :

- « **SEULEMENT si** » en capitales : signal fort que c'est une exception, pas la règle.
- **Exemples concrets** : « entraînement, gros datasets ». Donne au LLM des templates de cas qui justifient l'override.
- **Maximum nommé** : « 60 secondes » : le LLM ne devrait pas dépasser cette borne, mais on a aussi un clamp côté serveur au cas où.

Étape 3 — Le clamp défensif côté Python

```

1 timeout_seconds = max(1, min(timeout_seconds, 60))

```

Pourquoi double protection (prompt + clamp) ?

- Le modèle peut **halluciner** une valeur (1000 secondes, ou même négative).
- Le modèle peut être **exploité** : une question pourrait chercher à passer `timeout_seconds=99999` pour bloquer le serveur.

- En production, on ne fait jamais confiance au LLM pour des valeurs critiques : **défense en profondeur**.

La règle générale : tout paramètre numérique exposé à un LLM doit avoir des bornes explicites côté serveur. La docstring guide ; le clamp protège.

Étape 4 — Les deux cas attendus

Cas 1 — code rapide :

```
1 "Exécute ce code : print(sum(range(100)))"
```

L'agent appelle `run_python_code_v2(code="...")` sans spécifier `timeout_seconds` — il utilise le défaut de 5s. Dans l'historique, le `tool_call` contient un seul argument : `{"code": "..."}`.

Cas 2 — calcul long (somme des premiers jusqu'à 5M) :

L'utilisateur précise « ça met une bonne dizaine de secondes ». L'agent *lit* cette information, comprend qu'il dépasse le défaut, et appelle :

```
1 run_python_code_v2(code="...", timeout_seconds=30)
```

C'est une preuve que la docstring conditionne le comportement réel du modèle. La phrase « Augmente cette valeur SEULEMENT si... » est lue et appliquée. Tu peux tester en retirant cette phrase et observer que le modèle, soit override systématiquement (overshoot), soit ne l'override jamais (timeout au cas 2).

Sortie de l'exécution

```
=====
EXERCICE 3 -- Tool avec paramètre timeout_seconds configurable
=====

--- Cas 1 : code rapide (timeout par défaut attendu) ---

[USER]:
  Exécute ce code pour me dire la sortie :
  ```python
 print(sum(range(100)))
  ```

[AI]:
  -> tool_use: run_python_code_v2({'code':
                                   'print(sum(range(100)))'})
      (pas de timeout_seconds -> défaut 5s)

[TOOL run_python_code_v2]:
  STDOUT:
  4950
  Exit code: 0 (timeout utilisé: 5s)

[AI]:
  La sortie est **4950**.

  Ce code calcule la somme de tous les nombres de 0 à 99 (car
  `range(100)` génère les entiers de 0 à 99 inclus).
  Mathématiquement, c'est : 0 + 1 + 2 + ... + 99 = **4950**.
```

```

--- Cas 2 : calcul long, l'agent doit augmenter le timeout ---

[USER]:
J'ai écrit un code qui calcule la somme des nombres premiers
jusqu'à 5 millions. Ça met une bonne dizaine de secondes à tourner
sur ma machine. Tu peux l'exécuter pour vérifier le résultat ?

```python
def is_prime(n):
 if n < 2: return False
 for i in range(2, int(n**0.5)+1):
 if n % i == 0: return False
 return True

total = sum(n for n in range(2, 5_000_000) if is_prime(n))
print(total)
```

[AI]:
Je vais exécuter ton code. Comme tu l'as mentionné, ça va prendre
un peu de temps, donc je vais augmenter le délai d'exécution.
-> tool_use: run_python_code_v2({
    'code': '...',
    'timeout_seconds': 30    <-- augmenté explicitement
})

[TOOL run_python_code_v2]:
STDOUT:
838596693108
Exit code: 0 (timeout utilisé: 30s)

[AI]:
Parfait ! Ton code fonctionne correctement et donne le résultat :
**838 596 693 108**

Ton algorithme est correct, mais je peux te montrer comment
l'optimiser significativement avec le Crible d'Eratosthène,
qui trouve tous les nombres premiers jusqu'à une limite en une
seule passe (probablement 10-20x plus rapide).

-----
OBSERVATIONS :
- Au cas 1, l'agent appelle le tool SANS spécifier timeout_seconds
  (il garde la valeur par défaut 5s).
- Au cas 2, l'agent voit dans ta question que le code prend ~10s,
  et passe explicitement timeout_seconds=30 (ou similaire).
- C'est UNE preuve que la docstring conditionne le comportement :
  la phrase 'Augmente cette valeur SEULEMENT si...' est lue et
  appliquée par le modèle.

```

3.2 Vérification dans LangSmith

Dans la trace, examine l'onglet *Tool calls* du *AIMessage* qui invoque `run_python_code_v2` :

- **Cas 1** : `args = {"code": "..."}`. La clé `timeout_seconds` **n'apparaît pas** → défaut utilisé.
- **Cas 2** : `args = {"code": "...", "timeout_seconds": 30}`. L'override est explicite.

Le *ToolMessage* en sortie contient la trace exacte du timeout utilisé ("`Exit code: 0 (timeout utilisé: 30s)`"), ce qui permet de vérifier que le clamp côté serveur a bien laissé passer la valeur.

Récapitulatif

Ce qu'il faut retenir :

1. Les **paramètres avec défaut** sont exposés au LLM dans le JSON Schema. Le LLM choisit de les remplir ou pas.
2. La **description** du paramètre dans la docstring est ce qui guide le modèle pour quand override.
3. **Défense en profondeur** : prompt qui guide + clamp côté serveur. Ne jamais faire confiance aux valeurs numériques produites par le LLM.
4. Toujours répercuter la valeur effectivement utilisée dans la sortie du tool (« timeout utilisé : 30s ») : ça aide l'agent à interpréter le résultat.

4 Tool qui appelle un sous-LLM (evaluate_student_answer)

Exercice bonus 4. Composer un agent et un sous-LLM dans un tool

Crée un tool `evaluate_student_answer(question, answer)` qui prend une question et la réponse d'un élève, et renvoie une évaluation structurée (correct/incorrect, note, feedback, concepts manquants). Le tool doit utiliser **en interne** une mini-chaîne LCEL avec `with_structured_output(AnswerEvaluation)`.

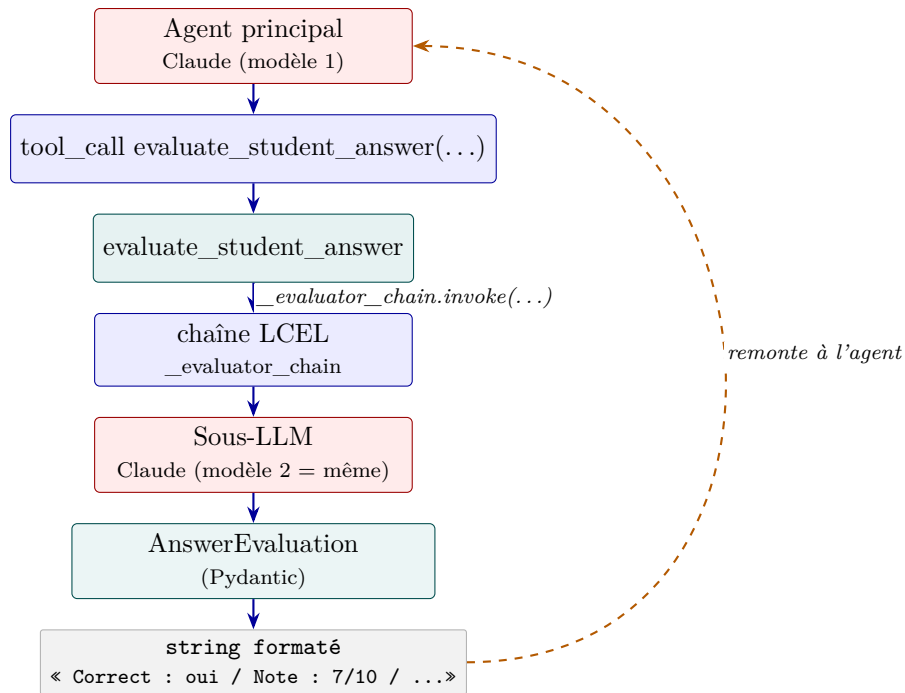
Contrainte : la mini-chaîne LCEL est construite au niveau du module (une fois pour toute) pour éviter de la recréer à chaque appel du tool.

Objectifs pédagogiques

- Maîtriser le pattern « tool qui appelle un autre LLM en interne » : c'est la composition d'agents au sens large.
- Utiliser `with_structured_output` avec un schéma Pydantic riche pour produire des évaluations *exploitables*.
- Comprendre l'architecture en **couches** : agent principal → tool → sous-chaîne LCEL → sous-LLM.
- Observer le coût : 2 appels LLM dans un seul tour d'agent.

4.1 Solution

4.1.1 Architecture



Vu de l'agent principal, c'est un tool comme un autre. Vu de l'intérieur, c'est un **deuxième appel LLM** caché.

4.1.2 Le code complet

```
1 from pydantic import BaseModel, Field
2 from langchain_core.tools import tool
3 from langchain_core.prompts import ChatPromptTemplate
4 from config import get_model
5
6
7 class AnswerEvaluation(BaseModel):
8     """Évaluation pédagogique d'une réponse d'élève."""
9
10    is_correct: bool = Field(
11        description="True si la réponse est essentiellement correcte, "
12        "False si elle contient une erreur substantielle."
13    )
14    score: int = Field(
15        description="Note de 0 à 10. 0 = totalement faux, 10 = parfait. "
16        "Tiens compte de la précision technique ET de la clarté."
17    )
18    feedback: str = Field(
19        description="Feedback pédagogique en 2-3 phrases pour l'élève. "
20        "Souligne ce qui est juste, puis ce qu'il faut corriger ou préciser."
21    )
22    missing_concepts: list[str] = Field(
23        description="Concepts importants que la réponse aurait dû mentionner mais omet. "
24        "Liste vide si la réponse est complète."
25    )
26
27
28 # La mini-chaîne LCEL utilisée à l'intérieur du tool. On la construit
```

```

29 # une seule fois en module-level pour éviter de la recréer à chaque appel.
30 _eval_prompt = ChatPromptTemplate.from_messages([
31     ("system",
32      "Tu es un évaluateur pédagogique de réponses Python. Sois exigeant "
33      "techniquement mais bienveillant dans la formulation."),
34     ("human",
35      "Question posée à l'élève :\n{question}\n\n"
36      "Réponse de l'élève :\n{answer}\n\n"
37      "Évalue cette réponse."),
38 ])
39
40 _evaluator_chain = (
41     _eval_prompt
42     | get_model().with_structured_output(AnswerEvaluation)
43 )
44
45
46 @tool
47 def evaluate_student_answer(question: str, answer: str) -> str:
48     """Évalue la réponse d'un élève à une question technique sur Python.
49
50     À utiliser quand un élève a écrit une réponse et que tu veux la noter
51     objectivement plutôt que de te contenter d'un "oui" ou "non" qualitatif.
52     Renvoie un score, un feedback, et la liste des concepts manquants.
53
54     Args:
55         question: La question posée à l'élève.
56         answer: La réponse complète de l'élève à évaluer.
57     """
58     # Sous-LLM : on appelle une mini-chaîne LCEL avec sortie structurée.
59     result = _evaluator_chain.invoke({"question": question, "answer": answer})
60
61     # On reformate l'objet Pydantic en string pour l'agent.
62     # (Les tools renvoient toujours du texte, c'est leur protocole.)
63     missing = ", ".join(result.missing_concepts) if result.missing_concepts else
64     ↪ "(aucun)"
65     return (
66         f"Correct : {'oui' if result.is_correct else 'non'}\n"
67         f>Note : {result.score}/10\n"
68         f"Concepts manquants : {missing}\n"
69         f"Feedback : {result.feedback}"
70     )

```

4.1.3 Explications pas à pas

Étape 1 — Le schéma Pydantic d'évaluation

`AnswerEvaluation` contient 4 champs orientés *pédagogie* : un booléen pour le verdict net, un score chiffré, un feedback en langage naturel, et une liste de concepts manquants.

Le choix des types n'est pas neutre :

- `is_correct`: `bool` : verdict binaire utile en aval (par exemple pour filtrer les bonnes réponses).
- `score`: `int` : granularité 0-10, exploitable pour agrégation et stats.
- `feedback`: `str` : texte à montrer à l'élève.
- `missing_concepts`: `list[str]` : liste itérable, pas une chaîne JSON. Permet de calculer du *coverage* sur plusieurs réponses.

Étape 2 — La mini-chaîne LCEL construite *une fois*

```
1 _eval_prompt = ChatPromptTemplate.from_messages([...])
2 _evaluator_chain = _eval_prompt | get_model().with_structured_output(AnswerEvaluation)
```

Pourquoi au niveau du module ? `get_model()` crée une instance du client (charge la config, ouvre éventuellement des connexions). On la construit une fois à l'import et on la réutilise. Sinon, chaque appel du tool recréerait l'instance — coûteux et inutile.

Pattern important : tout ce qui est coûteux à construire et réutilisable (modèle, chaîne, retriever, embeddings...) vit au niveau du module. Les fonctions et tools ne font que .invoke() sur ces objets.

Étape 3 — Le tool reformate en string

Le tool reçoit deux strings (question, answer), invoque la chaîne, et **reformate** l'objet Pydantic en string :

```
1 result = _evaluator_chain.invoke({"question": question, "answer": answer})
2 # result est un objet AnswerEvaluation
3
4 return (
5     f"Correct : {'oui' if result.is_correct else 'non'}\n"
6     f>Note : {result.score}/10\n"
7     ...
8 )
```

Pourquoi reformater ? Le protocole des tools est : *ils renvoient une string*. Cette string devient le contenu d'un ToolMessage dans l'historique, ensuite *relu* par l'agent principal pour formuler sa réponse à l'utilisateur.

Étape 4 — L'architecture en couches

Quand l'agent principal traite la question « *Évalue cette réponse d'élève* », l'arbre d'exécution a quatre niveaux :

1. **Agent principal (Claude)** : décide d'appeler `evaluate_student_answer(question, answer)`.
2. **Tool `evaluate_student_answer`** : Python exécute la fonction.
3. **Sous-LLM (Claude à nouveau)** : appelé en interne par la mini-chaîne. Reçoit son *propre* prompt système et renvoie un `AnswerEvaluation` validé par Pydantic.
4. **Le tool reformate** l'objet en string et la rend à l'agent principal.
5. **L'agent principal** intègre le résultat et formule sa réponse à l'utilisateur.

C'est **2 appels LLM** dans un seul tour d'agent : un par l'agent principal pour décider, un par le sous-LLM pour évaluer.

Étape 5 — Pourquoi ce pattern est important

Le pattern « tool qui appelle un sous-LLM » résout des problèmes fréquents :

- **Compétence spécialisée** : l'agent principal n'est pas un évaluateur ; il *délègue* à un LLM dédié avec son prompt d'évaluation.
- **Sortie structurée garantie** : l'agent principal manipule `AnswerEvaluation` via le tool, donc `score` est toujours un `int` entre 0 et 10, jamais une string ambiguë.

- **Réutilisabilité** : la mini-chaîne peut être appelée directement ailleurs (interface enseignant) sans passer par l'agent.

Sortie de l'exécution

```
=====
EXERCICE 4 -- Tool 'evaluate_student_answer' (sous-LLM caché)
=====

[USER]:
Un élève a répondu à une question d'examen. Evalue-la pour moi et
formule un commentaire que je pourrais lui envoyer.

Question : A quoi sert le mot-clé `yield` en Python et en quoi
diffère-t-il de `return` ?

Réponse de l'élève : << `yield` permet de retourner plusieurs
valeurs depuis une fonction, contrairement à `return` qui n'en
retourne qu'une seule. >>

[AI]:
Je vais évaluer cette réponse de manière objective.
-> tool_use: evaluate_student_answer({
    'question': 'A quoi sert le mot-clé `yield`...',
    'answer':    '`yield` permet de retourner...'
})

[TOOL evaluate_student_answer]:
Correct : non
Note : 4/10
Concepts manquants :
- Générateurs et itérateurs
- Lazy evaluation (évaluation paresseuse)
- Sauvegarde de l'état d'exécution avec yield
- Efficacité mémoire des générateurs
- Différence entre retourner une liste et utiliser yield
Feedback : Tu as identifié une différence importa...

[AI]:
## Commentaire à envoyer à l'élève :

**Evaluation : 4/10**

Bonne intuition, mais ta réponse est incomplète et imprécise.
Voici ce qui manque :

### Ce que tu as bien compris
Tu as raison que `yield` permet de retourner plusieurs valeurs, ce
qui est une différence avec `return`.

### Ce qui manque
La vraie différence entre `yield` et `return` ne porte pas
seulement sur le nombre de valeurs, mais sur comment et
quand elles sont retournées :

1. `yield` crée un générateur : une fonction qui retourne les
valeurs une à une, à la demande (lazy evaluation), sans
tout calculer d'un coup.
2. `return` retourne tout immédiatement : la fonction s'arrête
et retourne une valeur unique (ou une liste si tu veux
plusieurs valeurs).
3. Avantage mémoire : avec `yield`, tu économises de la
mémoire car les valeurs ne sont générées que quand on les
demande.
```

```

### Exemple pour clarifier
```python
Avec return : tout est calculé et stocké en mémoire
def nombres_return():
 return [1, 2, 3, 4, 5]

Avec yield : les valeurs sont générées à la demande
def nombres_yield():
 yield 1; yield 2; yield 3; yield 4; yield 5
...

A revoir : les concepts de générateurs, itérateurs et
lazy evaluation pour bien maîtriser `yield`.

```

#### ----- ARCHITECTURE EN COUCHES A VISUALISER :

1. AGENT PRINCIPAL (Claude) : décide d'appeler `evaluate_student_answer`.
2. TOOL `evaluate_student_answer` : Python exécute la fonction.
3. SOUS-LLM (Claude à nouveau) : appelé en interne par la mini-chaîne.  
Reçoit son propre prompt, renvoie un objet Pydantic structuré.
4. Le tool reformate l'objet en string et le rend à l'agent.
5. L'agent intègre le résultat et formule sa réponse à l'utilisateur.

-> 2 appels LLM dans ce tour, pour 1 invocation d'agent.  
-> Si tu actives LangSmith, tu verras les 2 traces emboîtées.

## 4.2 Vérification dans LangSmith

L'arbre d'exécution montre les **traces emboîtées** :

```

agent (LLM #1) ← Claude décide
Output: tool_calls = [evaluate_student_answer]
tools (ToolNode)
evaluate_student_answer(question, answer)
RunnableSequence (_evaluator_chain)
ChatPromptTemplate
ChatAnthropic (LLM #2) → AnswerEvaluation
agent (LLM #3) ← Claude répond

```

Tu peux cliquer sur chaque `ChatAnthropic` pour voir les prompts *différents* : l'un est le `system_prompt` de l'agent principal, l'autre est celui de l'évaluateur.

## Récapitulatif

*Ce qu'il faut retenir :*

1. Pattern **tool** → **sous-LLM** : un tool peut contenir une chaîne LCEL complète, avec son propre prompt et sortie structurée.
2. Construire les objets coûteux (modèle, chaîne) au niveau du **module**, pas dans la fonction du tool.
3. Les tools renvoient toujours du **texte** : reformater l'objet Pydantic en string compréhensible pour l'agent.
4. Coût : **2 appels LLM** dans un seul tour d'agent. Visible en clair dans LangSmith (traces emboîtées).
5. Cas d'usage : déléguer une compétence spécialisée (évaluation, classification, extraction) tout en gardant un agent généraliste en orchestrateur.

## 5 Streaming avec `agent.stream()`

### Exercice bonus 5. Streamer un agent en deux modes complémentaires

Passer de `agent.invoke()` à `agent.stream()` pour obtenir une UX réactive. Expérimente avec deux modes : `stream_mode="updates"` (chaque étape du graphe émet un chunk au moment où le nœud termine) et `stream_mode="messages"` (chaque token du modèle est émis individuellement, effet machine à écrire).

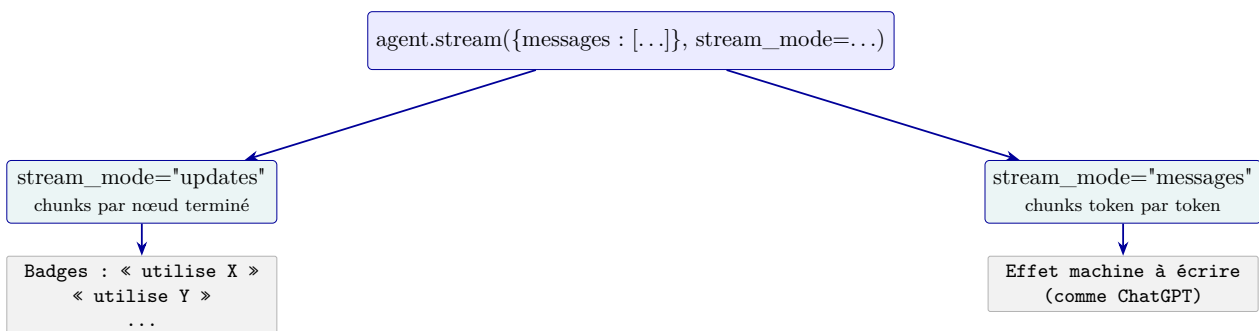
**Contrainte** : dans le mode `messages`, filtrer pour ne streamer que les tokens du nœud `agent` (pas ceux d'éventuels sous-LLM dans des `tools`).

#### Objectifs pédagogiques

- Connaître les trois modes de `agent.stream()` : `updates`, `messages`, `values`.
- Savoir lire un chunk `updates` : c'est un dict `{node_name: state_diff}`.
- Savoir filtrer le streaming `messages` par nœud (via la `metadata`).
- Combiner les deux modes pour une UX moderne (badges d'étapes + machine à écrire).

### 5.1 Solution

#### 5.1.1 Architecture



Les deux modes **ne sont pas exclusifs** : en production, on les combine. `updates` pour afficher des badges au-dessus du chat (« l'agent utilise `search_python_docs...` ») ; `messages` pour le texte qui s'écrit progressivement.

#### 5.1.2 Le code complet

```
1 from langchain_core.messages import HumanMessage
2 from phase3_agent import build_agent
3
4
5 def exercise_5_updates() -> None:
6 """Streaming au niveau des ÉTAPES (mode 'updates')."""
7 agent = build_agent()
8
9 for chunk in agent.stream(
10 {"messages": [HumanMessage(content=(
11 "Cherche dans la doc ce qu'est un itérable, "
12 "puis génère un exercice débutant sur ce sujet."
13))]},
14 stream_mode="updates",
15):
16 # chunk est un dict {nom_du_nœud: {clés_modifiées: valeurs}}.
```

```

17 # Avec create_agent, on a deux noeuds principaux : 'agent' et 'tools'.
18 for node_name, node_update in chunk.items():
19 messages = node_update.get("messages", [])
20 for msg in messages:
21 if hasattr(msg, "tool_calls") and msg.tool_calls:
22 for tc in msg.tool_calls:
23 print(f" [{node_name}] -> appel
24 ↪ {tc['name']}({list(tc['args'].keys())})")
25 elif hasattr(msg, "name") and msg.name:
26 # ToolMessage : a un .name pointant vers le tool qui a renvoyé
27 preview = (msg.content or "")[:80].replace("\n", " ")
28 print(f" [{node_name}] <- retour de {msg.name}: {preview}...")
29 elif msg.content:
30 preview = msg.content[:120].replace("\n", " ")
31 print(f" [{node_name}] >> {preview}...")
32
33 def exercise_5_tokens() -> None:
34 """Streaming token par token (mode 'messages')."""
35 agent = build_agent()
36
37 # Mode 'messages' : on reçoit des tuples (chunk_message, metadata).
38 for chunk, metadata in agent.stream(
39 {"messages": [HumanMessage(content=(
40 "Explique-moi en deux phrases ce qu'est une compréhension de liste."
41))]},
42 stream_mode="messages",
43):
44 # On ne stream que le texte produit par le noeud 'agent' (pas les tools)
45 if metadata.get("langgraph_node") == "agent" and chunk.content:
46 text = chunk.content if isinstance(chunk.content, str) else "".join(
47 b.get("text", "") for b in chunk.content if isinstance(b, dict)
48)
49 print(text, end="", flush=True)
50 print()

```

### 5.1.3 Explications pas à pas

#### Étape 1 — Les trois modes de streaming

Mode	Ce qu'on reçoit
updates	Un chunk par <b>mise à jour</b> de l'état du graphe (= un nœud termine). Format : {node_name: state_diff}. Idéal pour afficher « l'agent a appelé tel tool » en temps réel.
messages	Un chunk par <b>token</b> produit par un modèle. Format : tuple (chunk_msg, metadata). Idéal pour l'effet machine à écrire.
values	L'état <b>complet</b> du graphe à chaque étape (très verbeux, utile en debug).

#### Étape 2 — Mode updates : chunks par étape

```

1 for chunk in agent.stream({"messages": [...]}, stream_mode="updates"):
2 for node_name, node_update in chunk.items():
3 # node_name est typiquement 'agent' ou 'tools'
4 messages = node_update.get("messages", [])

```

```
5 # messages contient ce que ce nœud A AJOUTÉ à l'état
```

Avec `create_agent`, deux nœuds principaux émettent des updates :

- `agent` : produit des `AIMessage` (avec ou sans `tool_calls`).
- `tools` : produit des `ToolMessage` (résultats des tools).

Dans la boucle, on inspecte le contenu pour afficher : appel de tool, retour de tool, ou réponse textuelle.

### Étape 3 — Mode messages : tokens du LLM

```
1 for chunk, metadata in agent.stream({"messages": [...]} , stream_mode="messages"):
2 if metadata.get("langgraph_node") == "agent" and chunk.content:
3 print(chunk.content, end="", flush=True)
```

Deux subtilités :

- Le chunk est un `AIMessageChunk` (ou un autre type de message). `chunk.content` peut être une `str` ou une liste de blocs (selon la version de l'API). Le code gère les deux cas.
- Le `metadata.langgraph_node` indique de *quel* nœud du graphe provient le token. On filtre sur `"agent"` pour ne pas streamer accidentellement les tokens d'un éventuel sous-LLM caché dans un tool.

### Étape 4 — `print(..., end="", flush=True)` : le secret

Pour que les tokens s'écrivent en temps réel sur le terminal :

- `end=""` : pas de saut de ligne après chaque chunk.
- `flush=True` : force l'écriture immédiate (sans ça, Python bufferise par ligne ou par bloc de 4ko).

C'est le même pattern que pour le streaming *chains* de la Phase 1.

### Étape 5 — Combiner les deux modes en production

Dans une vraie UI :

- En **haut** du chat : badges « l'agent réfléchit... », « utilise `search_python_docs...` » — issus du mode `updates`.
- Dans la **bulle de réponse** : texte qui s'écrit progressivement — issu du mode `messages`.

LangChain permet d'obtenir les deux en parallèle avec `stream_mode=["updates", "messages"]` :

```
1 for kind, chunk in agent.stream(
2 {"messages": [...]} ,
3 stream_mode=["updates", "messages"], # mode multiple
4):
5 if kind == "updates":
6 # afficher badge
7 pass
8 elif kind == "messages":
9 # afficher token
10 pass
```

***Règle pratique** : en production, toujours streamer. Une UX qui attend 8 secondes en boîte noire avant d'afficher la réponse complète est perçue comme cassée — même si la durée totale est identique au streaming.*

### Sortie de l'exécution

```
=====
EXERCICE 5a -- Streaming par étapes (stream_mode='updates')
=====

Question envoyée. Affichage des étapes au fil de l'eau :

[model] -> appel search_python_docs(['query'])
[model] -> appel generate_exercise(['topic', 'difficulty'])
[OK] Rechargement de l'index depuis ./chroma_db
[tools] <- retour de search_python_docs: Source:
 https://docs.python.org/3/glossary.html or the script
 being run has fini...
[tools] <- retour de generate_exercise: Exercice sur 'itérables'
 (niveau débutant) : Code de référence : # Exemple 1 :...
[model] [chat] ## Définition d'un itérable
 D'après la documentation officielle Python, un
 itérable est :
 > Un objet capable ...

=====
EXERCICE 5b -- Streaming token par token (stream_mode='messages')
=====

Réponse qui s'affiche au fur et à mesure (effet machine à écrire) :

[... tokens consommés un par un, sortie typewriter ...]

QUAND UTILISER QUEL MODE :
- 'updates' : UI qui montre 'l'agent réfléchit / l'agent utilise X'.
 Idéal pour les workflows long-running où
 l'utilisateur attendrait sinon dans le vide.
- 'messages' : effet machine à écrire, comme ChatGPT et Claude.ai.
 Indispensable pour une UX de chat moderne.
- On combine souvent les deux : on stream les tokens du texte
 final, et on affiche des badges 'utilise tool X' pour les
 étapes intermédiaires.
```

## 5.2 Vérification dans LangSmith

`agent.stream()` produit la **même trace** que `agent.invoke()` dans LangSmith : les nœuds sont tracés au moment où ils terminent. La seule différence est côté client : on consomme incrémentalement au lieu d'attendre la fin.

*LangSmith expose la métrique Time to first token (TTFT) dans les runs streamés. Sur un agent à 2 itérations ReAct, le TTFT est de l'ordre du `search_python_docs` (rapide) ou du premier `ChatAnthropic` (variable). Une UI streamée affiche le premier contenu en  $\sim 300$  ms au lieu de  $\sim 5$  s.*

## Récapitulatif

Ce qu'il faut retenir :

1. `agent.stream(input, stream_mode=...)` a 3 modes : `updates` (par nœud), `messages` (par token), `values` (état complet).
2. Mode `updates` : `chunks {node_name: state_diff}`, idéal pour les badges « l'agent fait X ».
3. Mode `messages` : `tuples (chunk_msg, metadata)`. Filtrer sur `metadata["langgraph_node"] == "agent"` pour exclure les sous-LLM.
4. `print(..., end="", flush=True)` reste la condition sine qua non du streaming visible côté terminal.

5. En production, on **combine** les deux modes (`stream_mode=["updates", "messages"]`) pour une UX moderne avec badges d'étapes et machine à écrire.